Binary Search - Pre-requisite : Sorted Data : Divide and conquer technique

12 April 2026 20:00

Key : 77

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
11	22	33	44	55	66	77	88	99
Left				Mid				Right

Left Sub array

Right Sub array.

1) Get the key from the user.
2) Divide your array in two parts.
   Find the left index,right index and the mid index.
   Left : 0
   Right : SIZE - 1 : 8
   Mid : (Left + Right) / 2
   (0+8) / 2 : 4

3) Compare the key with the element at the mid index.
   Key == arr[mid]
   77 == 55 ? NO

   If the key is matching the element at mid, return the corresponding index,
   Else go to step 4.

4) Check if the key is smaller to the mid element or greater ?
   If the key is smaller , then continue the search in the left sub-array.
   Left sub-array is from Left index to Mid-1 Index.

   If the key is greater, then continue the search in the right sub-array.
   Right Sub-array is From Mid+1 index to Right index.

mid+1 Right

[5]	[6]	[7]	[8]
66	77	88	99
Left	Mid		Right

As the key is greater to the mid element, continue the search in the rigth sub- array.
Continue steps 2 and 3.

2) Find the left index, right index and the mid index.
Left = 5 :(mid+1)
Right = 8
Mid = (left+right) / 2
(5+8)/2 --> 6

3) Compare the key with the element at mid index.
   Key == arr[mid]
   77 == 77 ? Yes
   Key found at index 6

Best case time complextiy : if the key is found at 1st comparison : for eg : key 55 :
O(1)

Worst case time complexity : if the key is at extreme ends (left end or right end) of the array/ key not found.
In that case in half comparsions our job is done, hence
Worst case time comp : O(log n)

Avg case time complexity :
O(log n/2), here 1/2 nedds to be discarded.
Hence avg case : O(log n)

Input space : input array : O(n)

Aux space : O(1) as the extra variables required is constant.